

计算机体系结构

09. 相关与冒险

李建华

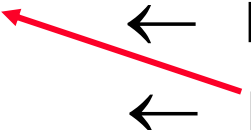
计算机与信息学院

The contents of the slides are adapted from CA course of wisc, princeton, mit, berkeley, edinburgh, and eth.
The uses of the slides of this course are for educational purposes only and should be used only in conjunction with the textbook. Derivatives of the slides must acknowledge the copyright notices of this and the originals.

回顾：数据相关的类型

流相关

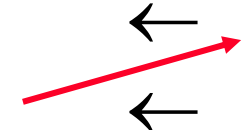
$r_3 \leftarrow r_1 \text{ op } r_2$
 $r_5 \leftarrow r_3 \text{ op } r_4$



Read-after-Write
(RAW)

反相关

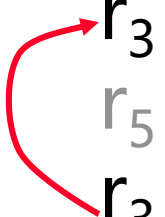
$r_3 \leftarrow r_1 \text{ op } r_2$
 $r_1 \leftarrow r_4 \text{ op } r_5$



Write-after-Read
(WAR)

输出相关

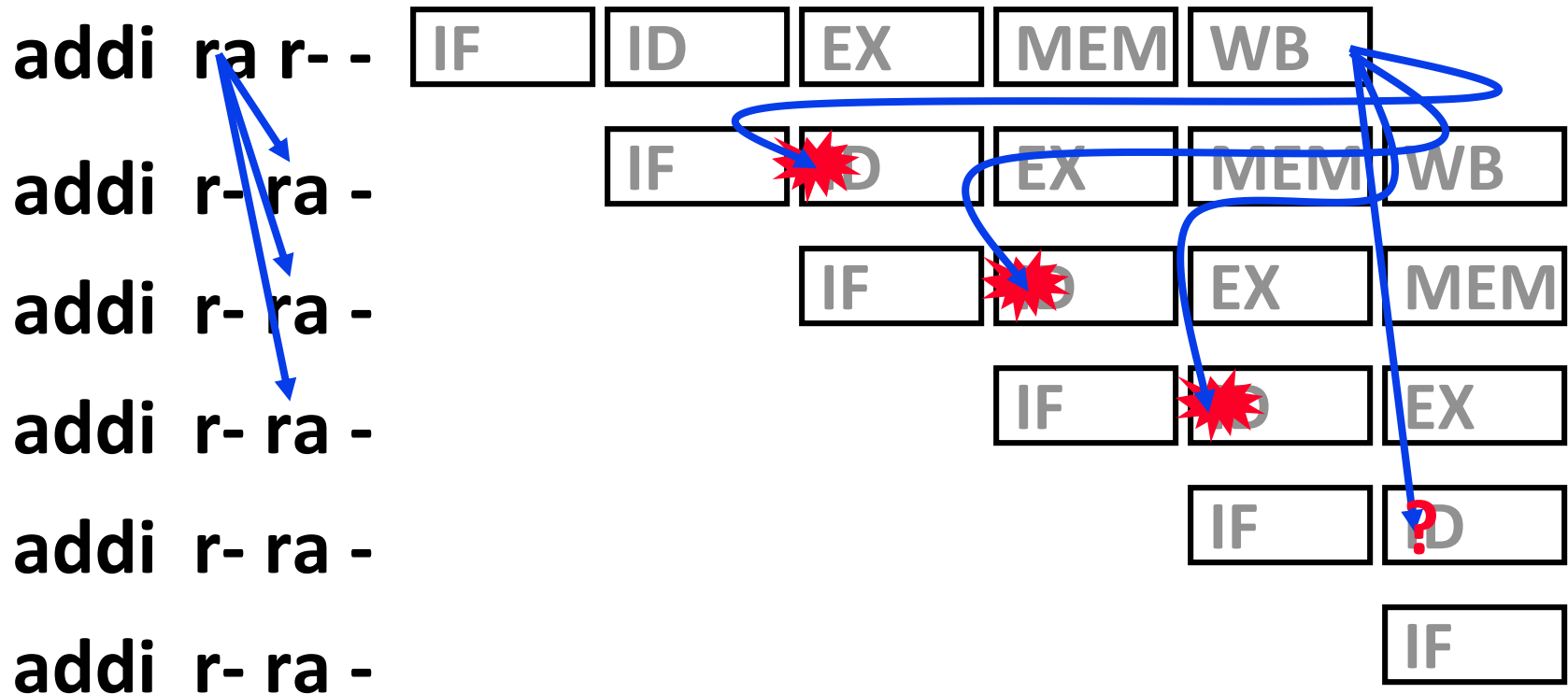
$r_3 \leftarrow r_1 \text{ op } r_2$
 $r_5 \leftarrow r_3 \text{ op } r_4$
 $r_3 \leftarrow r_6 \text{ op } r_7$



Write-after-Write
(WAW)

真相关的处理

- 下面所列的真相关,在前面介绍的5阶段流水线中哪些会导致冒险?

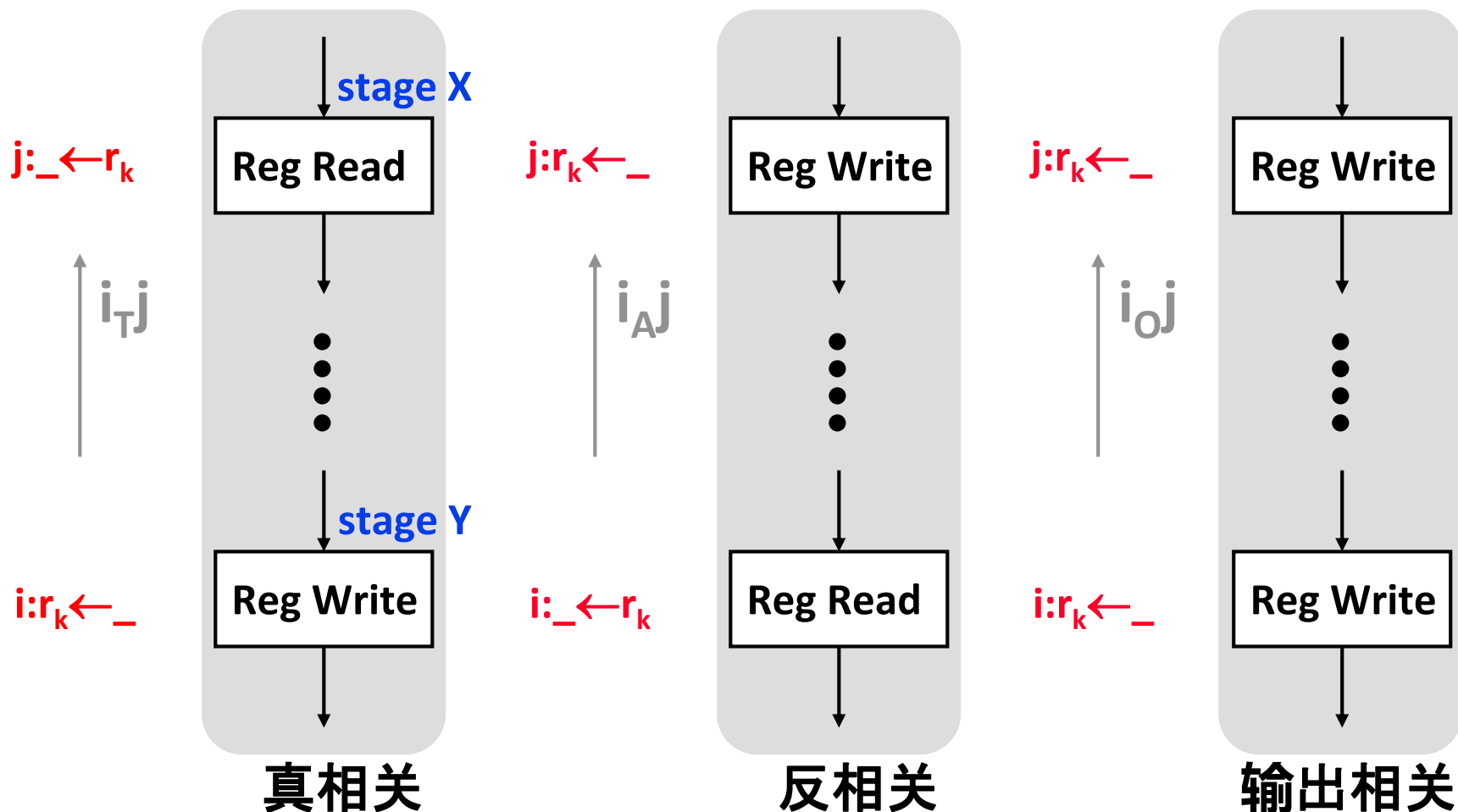


相关是否导致流水线冒险的分析

	R/I-Type	LW	SW	Br	J	Jr
IF						
ID	read RF	read RF	read RF	read RF		read RF
EX						
MEM						
WB	write RF	write RF				

- 对于给定的流水线来说，二条相关的指令是否会导致流水线冒险（Hazard）受哪些因素影响？
 - 相关的类型: RAW, WAR, WAW?
 - 相关的指令的具体类型?
 - 相关的指令之间的距离?

安全/不安全的流水线推进



$\text{dist}(i,j) \leq \text{dist}(X,Y) \Rightarrow$ 让j向前流动不安全
 $\text{dist}(i,j) > \text{dist}(X,Y) \Rightarrow$ 让j向前流动安全

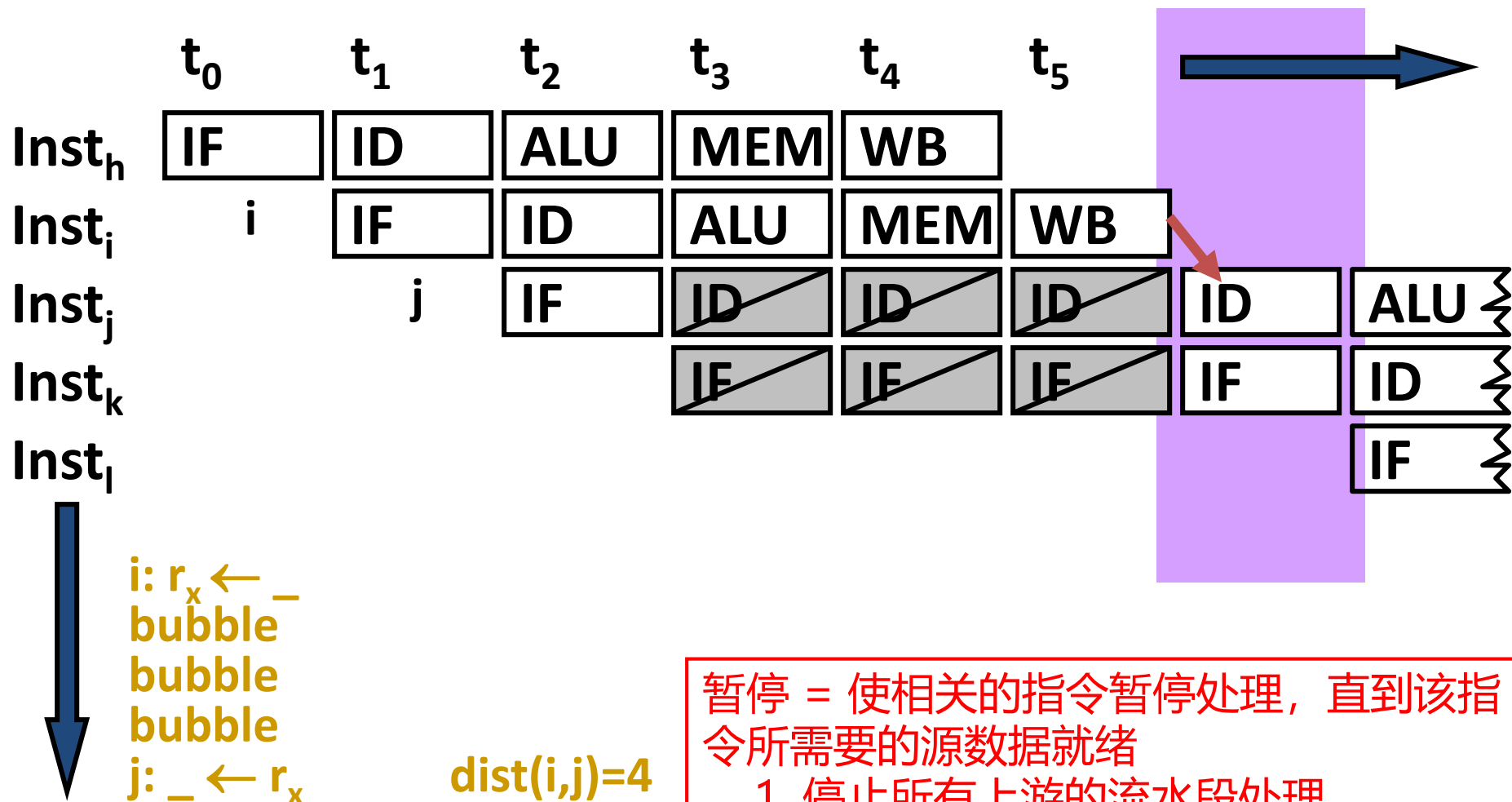
MIPS 5级流水线真相关的冒险分析

	R/I-Type	LW	SW	Br	J	Jr
IF						
ID	read RF	read RF	read RF	read RF		read RF
EX						
MEM						
WB	write RF	write RF				

- 指令 I_A 和 I_B (I_A 在 I_B 之前) 真相关导致流水线冒险, 当且仅当:
 - I_B (R/I, LW, SW, Br or JR) 读了一个被 I_A (R/I or LW) 写的寄存器 (目标寄存器)
 - 且 $\text{dist}(I_A, I_B) \leq \text{dist}(\text{ID}, \text{WB}) = 3$

WAR和WAW相关可以采用类似的分析方法

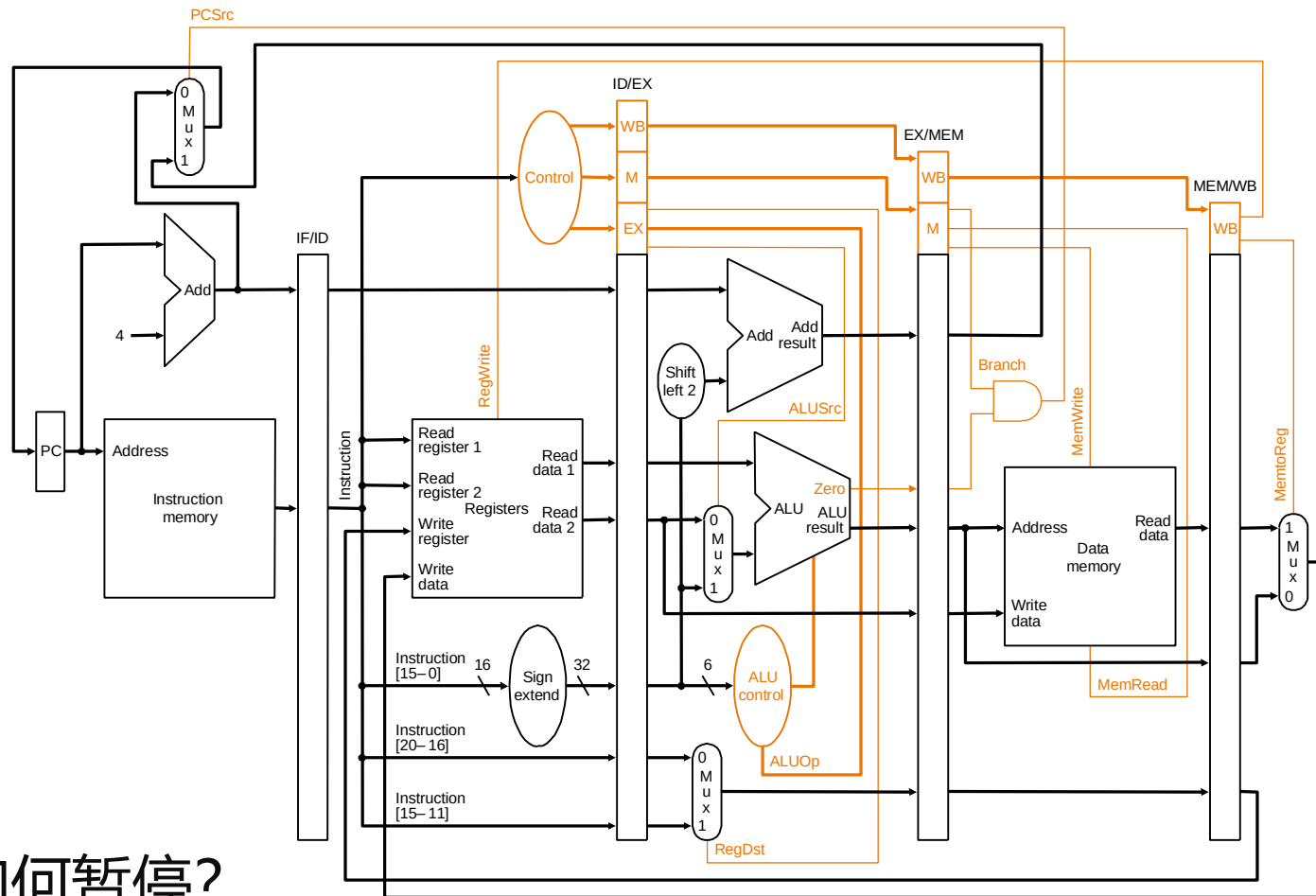
通过流水线暂停解决冒险



暂停 = 使相关的指令暂停处理，直到该指令所需要的源数据就绪

1. 停止所有上游的流水段处理
2. 排干所有下游的流水段任务

如何实现流水线暂停



- 如何暂停?

- 关闭**PC**和**IR** 的锁存; 确保被暂停的指令停留在其原来的阶段。
- 向被阻塞阶段的后续流水段插入 **空指令/nops** (叫做 “bubbles”)

暂停条件

- 真相关指令 I_A 和 I_B (I_A 在 I_B 之前) 导致流水线冒险, 当且仅当:
 - I_B (R/I, LW, SW, Br or JR) 读了一个被 I_A (R/I or LW) 写的寄存器
 - $\text{dist}(I_A, I_B) \leq \text{dist}(\text{ID}, \text{WB}) = 3$
- 当处于ID段的指令 I_B 想读一个处于EX, MEM, 或者WB段的指令 I_A 想要写的寄存器, 那么必须暂停ID段的处理。

暂停条件的检测逻辑

- 辅助函数：
 - $rs(I)$ 返回指令 I 的 rs 域
 - $rt(I)$ 返回指令 I 的 rt 域
- 当下面任何一个条件成立均需暂停：
 - $(rs(IR_{ID}) == dest_{EX}) \ \&\& \ RegWrite_{EX}$
 - $(rs(IR_{ID}) == dest_{MEM}) \ \&\& \ RegWrite_{MEM}$
 - $(rs(IR_{ID}) == dest_{WB}) \ \&\& \ RegWrite_{WB}$
 - $(rt(IR_{ID}) == dest_{EX}) \ \&\& \ RegWrite_{EX}$
 - $(rt(IR_{ID}) == dest_{MEM}) \ \&\& \ RegWrite_{MEM}$
 - $(rt(IR_{ID}) == dest_{WB}) \ \&\& \ RegWrite_{WB}$

特殊的\$0

暂停流水线的影响

- 每个暂停周期相当于浪费了一个周期，因为该周期完成的是空指令。
- 若一个程序执行了N条指令，暂停了S 个时钟周期, 那么：平均CPI $\approx (N+S)/N$
- 对MIPS 5段流水线来说，S的大小依赖于：
 - 真相关 发生的频率
 - 真相关的指令之间的距离
 - 影响暂停的时钟周期数

示例代码 (P&H)

- for (j=i-1; j>=0 && v[j] > v[j+1]; j-=1) { }

for2tst: addi \$s1, \$s0, -1 3 stalls
 slti \$t0, \$s1, 0 3 stalls
 bne \$t0, \$zero, exit2
 sll \$t1, \$s1, 2 3 stalls
 add \$t2, \$a0, \$t1 3 stalls
 lw \$t3, 0(\$t2)
 lw \$t4, 4(\$t2) 3 stalls
 slt \$t0, \$t4, \$t3 3 stalls
 beq \$t0, \$zero, exit2

 addi \$s1, \$s1, -1
 j for2tst

多少个
暂停周期?

exit2:

用数据前推减少暂停

- 又叫数据旁路 (Data Bypassing)
- 当指令所需的操作数 (**值**) 可用时, 尽快将其送给 (forward) 相关的指令。
- 数据前推的原理和**数据流模型**类似
 - 数据值**可用后**尽快传给**需要该值**的指令 (相关的指令)
 - 不是通过寄存器作为媒介来传递值
 - 在数据流模型下, 一条指令所需的所有操作数就绪后就可以执行。
 - 对dataflow model感兴趣的同学, 可以自行调研。
 - 可以作为课程报告的主题

用数据前推解决真相关造成的冒险

- 如果真相关的指令 I_A 与 I_B 造成冒险, 则处于 ID 段的 I_B 读了一个处于 EX、MEM 或者 WB 阶段的指令 I_A 要写的目标寄存器; **而此时指令 I_B 所需要的操作数的值还不在 I_B 要读取的源寄存器里面。**

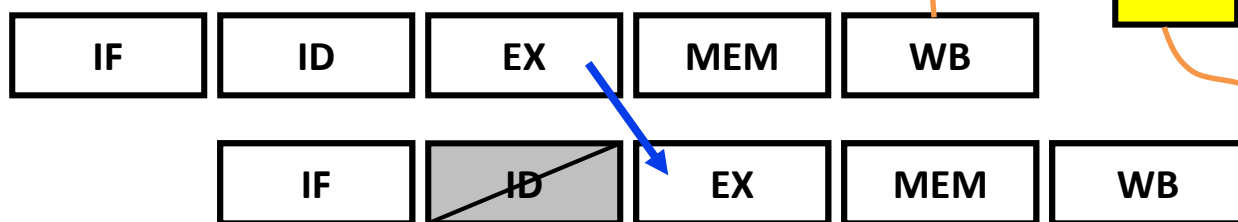
此时的目标操作数在哪里?

- 数据前推如何工作?

⇒ 从 datapath 中获取操作数(值), 而不是从 RF 中获取。

add rz r- r-

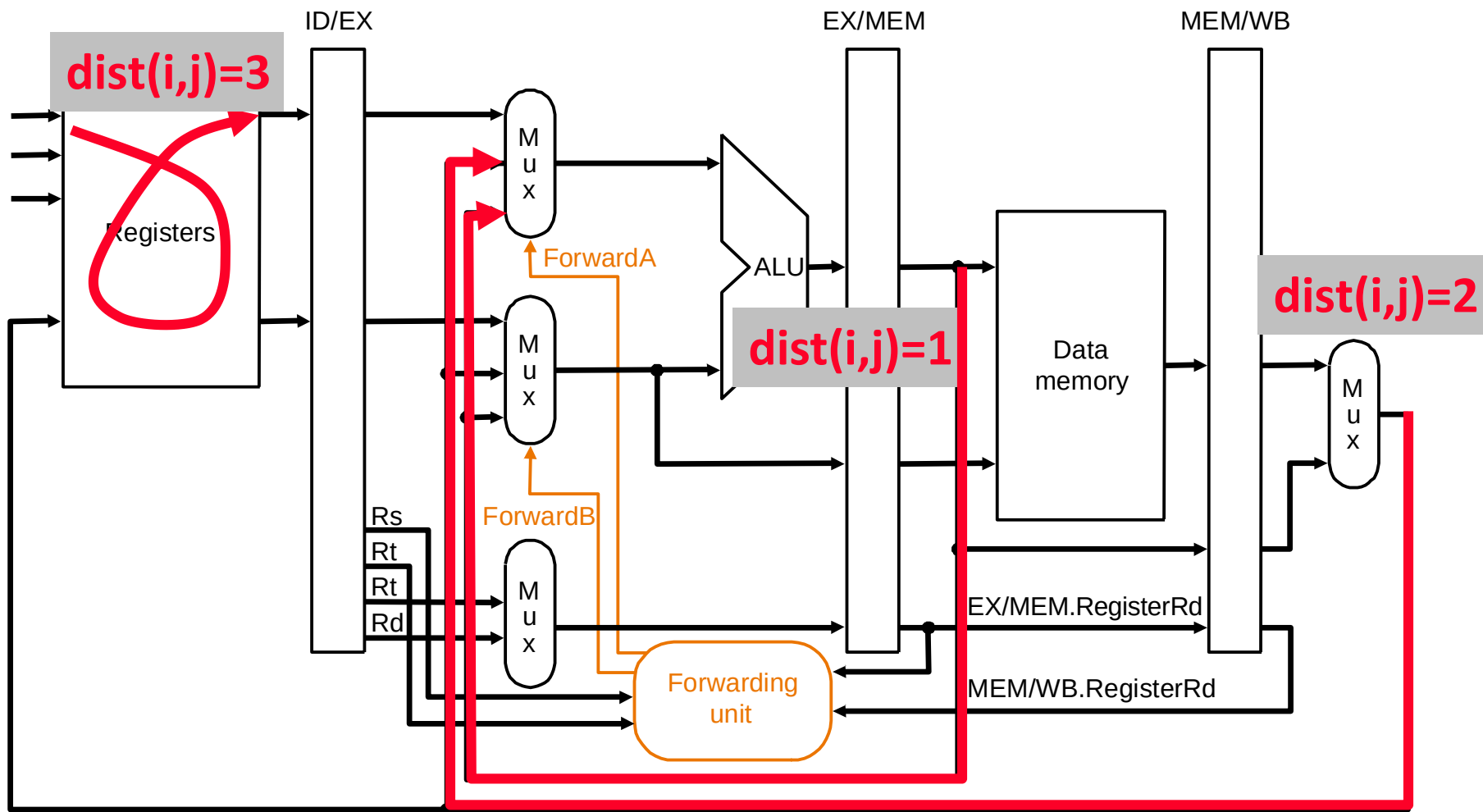
addi r- rz r-



⇒ **注意:** 从**潜在的多个定义值**中, 获取**最新的**操作数。

二者缺一不可!

数据前推的路径



b. With forwarding

寄存器文件进行内部前推

数据前推的逻辑

```
if ( $rs_{EX} \neq 0$ ) && ( $rs_{EX} == dest_{MEM}$ ) &&  $RegWrite_{MEM}$  then  
    forward operand from MEM stage // dist=1  
else if ( $rs_{EX} \neq 0$ ) && ( $rs_{EX} == dest_{WB}$ ) &&  $RegWrite_{WB}$  then  
    forward operand from WB stage // dist=2  
else  
    use operand from register file // dist >= 3
```

→注意: 

- ① 可能有多个潜在的前推对象，顺序很关键！
- ② 须从最新的潜在对象进行检测，匹配即前推！

有数据前推后的冲突分析

	R/I-Type	LW	SW	Br	J	Jr
IF						
ID						use
EX	use produce	use	use	use		
MEM		produce	(use)			
WB						

- 数据前推无法消除所有的数据相关造成的流水线冒险；
- 紧随LW指令的真相关仍然会导致流水线冒险。因此，需要暂停一个周期。

示例代码，无数据前推 (P&H)

- for (j=i-1; j>=0 && v[j] > v[j+1]; j-=1) { }

```
for2tst:  addi    $s1, $s0, -1           3 stalls
          slti    $t0, $s1, 0         3 stalls
          bne     $t0, $zero, exit2
          sll     $t1, $s1, 2         3 stalls
          add     $t2, $a0, $t1       3 stalls
          lw      $t3, 0($t2)
          lw      $t4, 4($t2)         3 stalls
          slt     $t0, $t4, $t3       3 stalls
          beq     $t0, $zero, exit2
          .....
          addi    $s1, $s1, -1
          j       for2tst
exit2:
```

示例代码，有数据前推 (P&H)

- for (j=i-1; j>=0 && v[j] > v[j+1]; j-=1) { }

```
                addi    $s1, $s0, -1
for2tst:        slti    $t0, $s1, 0
                bne     $t0, $zero, exit2
                sll     $t1, $s1, 2
                add     $t2, $a0, $t1
                lw      $t3, 0($t2)
                lw      $t4, 4($t2)
                nop
                slt     $t0, $t4, $t3
                beq     $t0, $zero, exit2
                .....
                addi    $s1, $s1, -1
                j        for2tst
exit2:
```

回顾: 控制相关

- **问题:** 对于流水线来说, 下一个周期要取指所依赖的PC是什么?
- **答案:** 下一条指令的地址
 - 那么, 所有的指令均控制相关于它前面的指令。 为什么?
- **如果**当前取到的指令是非控制流指令:
 - 下一个PC就是顺序的下一条指令的地址, 如 $PC+4$
 - 只需要知道指令的宽度, 很容易可以确定
- **如果**当前取到的指令是控制流指令:
 - 如何确定下一个PC呢?
- **挑战:** 实际上, 我们如何/何时才能知道当前取到的指令是否是控制流指令呢?

不同类型分支的属性

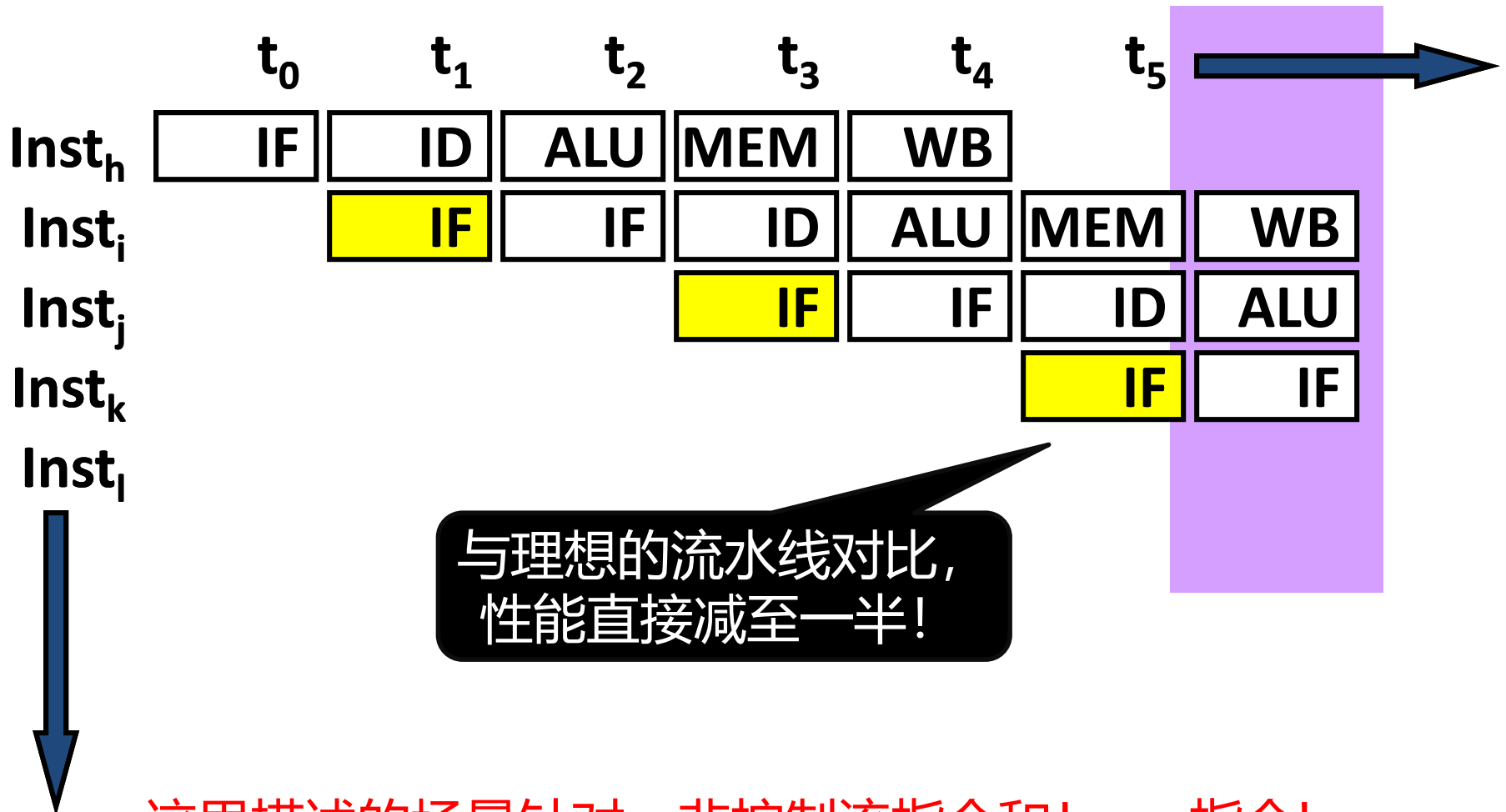
Type	Direction at fetch time	Number of possible next fetch addresses?	When is next fetch address resolved?
Conditional	Unknown	2	EXE (register dependent)
Unconditional	Always taken	1	ID (PC + offset)
Call	Always taken	1	ID (PC + offset)
Return	Always taken	Many	EXE (register dependent)
Indirect	Always taken	Many	EXE (register dependent)

不同分支类型的处理方法不一样

如何处理可能的暂停？

- 关键问题是：如何用**正确的动态指令序列**使得流水线能稳定运行
- 如果一条指令是控制流指令，那么潜在的解决方案有：
 - 暂停流水线，直到**明确**下一条指令的地址；
 - 猜测下一条指令的地址 (branch prediction)
 - 使用延迟分支技术 (branch delay slot)
 -

暂停流水线分析



这里描述的场景针对：非控制流指令和Jump指令!

比暂停更好的一个方案...

- 与其等待依赖于PC的真相关消除, 不如**猜测**
 $\text{nextPC} = \text{PC} + 4$, 并直接进行取指操作。
 - 这是一种好的猜测方法吗?
 - 如果猜错了, 会造成什么后果?
- ~20% 的指令组合是控制流指令
 - ~50 % 的向前跳转(i.e., if-then-else) 是跳转的
 - ~90% 的后向跳转(i.e., loop back) 是跳的
 - 总的来说: ~70% taken and ~**30%** not taken
[Lee and Smith, 1984]
- 因此, 猜测 “ $\text{nextPC} = \text{PC} + 4$ ” 的正确率
~**86%**, 那么14%猜错的情况如何处理?
 - $80\% + 20\% * 30\%$

Branch Prediction

- 总是猜测顺序的下一条指令就是即将执行的指令
- 这是一种分支预测机制 (branch prediction)
 - 后续会专门介绍分支预测机制!
- 请大家思考： **如何使得上面的方案更加高效?**

The screenshot shows the IEEE Xplore search results page. The top navigation bar includes links for IEEE.org, IEEE Xplore, IEEE SA, IEEE Spectrum, and More Sites, along with buttons for Subscribe, Donate, Cart, and Create Account. The main header features the IEEE Xplore logo, navigation links (Browse, My Settings, Help), and an Institutional Sign In button. A search bar with a dropdown menu set to 'All' and a search icon is present, with an 'ADVANCED SEARCH' link below it. The results section shows 'Showing 1-25 of 7,274 results for branch prediction'. Below this, there are filters for different document types: Conferences (5,304), Journals (1,766), Magazines (120), Early Access Articles (70), Books (11), and Standards (3). Each filter has an unchecked checkbox.

IEEE.org | IEEE Xplore | IEEE SA | IEEE Spectrum | More Sites

Subscribe | Donate | Cart | Create Account

IEEE Xplore® Browse ▾ My Settings ▾ Help ▾ Institutional Sign In

All ▾

ADVANCED SEARCH

Search within results

Items Per Page ▾ Export Set Search Alerts

Showing 1-25 of 7,274 results for **branch prediction**×

☐ Conferences (5,304) ☐ Journals (1,766) ☐ Magazines (120) ☐ Early Access Articles (70)

☐ Books (11) ☐ Standards (3)

Next Topic

3.1	Instruction-Level Parallelism: Concepts and Challenges	148
3.2	Basic Compiler Techniques for Exposing ILP	156
3.3	Reducing Branch Costs with Advanced Branch Prediction	162
3.4	Overcoming Data Hazards with Dynamic Scheduling	167
3.5	Dynamic Scheduling: Examples and the Algorithm	176
3.6	Hardware-Based Speculation	183
3.7	Exploiting ILP Using Multiple Issue and Static Scheduling	192
3.8	Exploiting ILP Using Dynamic Scheduling, Multiple Issue, and Speculation	197
3.9	Advanced Techniques for Instruction Delivery and Speculation	202